PERSONALISED ECHO CARE

A MINI PROJECT REPORT FOR THE COURSE

CS23621 – MOBILE APPLICATION DEVELOPMENT LABORATORY

Submitted by

PRASHAANT V (231001151)

NITHIN KUMAR (231001143)

III YEAR B.Tech

in

INFORMATION TECHNOLOGY

DEPARTMENT OF INFORMATION TECHNOLOGY

RAJLAKSHMI ENGINEERING COLLEGE

THANDALAM, CHENNAI 602 105

MAY 2026

BONAFIDE CERTIFICATE

Certified that the mini project titled **"PERSONALISED ECHO CARE"** is the bonafide work carried out by **Prashaant V(231001151) and Nithin Kumar P B(231001143)** under my supervision. This project has been completed in partial fulfilment of the requirements of the course CS23621-Mobile Application Development Laboratory, promoting project- based learning and practical application of design principles.

Certified further that, to the best of my knowledge and belief, the work reported herein is original and does not form part of any other project, report, or work submitted previously for any academic award.

This project contributes toward achieving the following **Sustainable Development Goals (SDGs):**

1. Good Health and Well-being (SDG 3)
2. Industry, Innovation and Infrastructure (SDG 9)
3. Reduced Inequalities (SDG 10)

Signature of the Supervisor with date

Signature Examiner-1

Signature Examiner-2

TABLE OF CONTENTS

ABSTRACT

InclusiveCare is an innovative, AI-powered Android application designed to bridge the critical communication and accessibility gap faced by individuals with speech, hearing, and visual impairments. In an era where mobile technology pervades every facet of daily life, persons with disabilities continue to encounter significant barriers to digital participation, independent living, and effective communication. Existing assistive solutions are largely fragmented — designed for a single disability type, often lacking real-time responsiveness, emergency integration, or multi-modal support — leaving users to manage multiple applications with inconsistent interfaces.

InclusiveCare addresses this gap comprehensively by integrating six specialised modules into a single, unified platform: a Blind Mode featuring ML Kit-based obstacle detection, OCR text reading, currency recognition, GPS navigation, and emergency SOS; a Hearing Mode providing real-time environmental sound detection powered by the YAMNet deep learning model along with an AI-powered chat interface; a Speech Mode enabling text-to-speech communication, quick phrase buttons, emotion expression, and word prediction; a Sign Language Mode leveraging MediaPipe hand landmark detection for gesture-based communication; a Home Mode serving as the central navigation hub; and a Road Navigation Mode offering voice-guided outdoor navigation assistance.

The application is architected on Firebase Authentication and Firestore for secure, cloud-based user management. It leverages Google ML Kit for real-time image processing, TensorFlow Lite (YAMNet) for on-device audio classification across 521 sound categories, CameraX for live camera feed processing, and MediaPipe for hand gesture recognition. The Android frontend is built using Kotlin and Java within Android Studio, with supplementary APIs including Android Text-to-Speech (TTS), SpeechRecognizer, Android LocationManager, Geocoder, and Google Maps Intent for navigation.

The development process followed the Stanford Design Thinking methodology — Empathize, Define, Ideate, Prototype, and Test — ensuring that every design decision was rooted in genuine user needs, validated through real-world testing, and refined iteratively based on concrete feedback. The outcome is a practical, empathetic, and technically robust platform that promotes digital accessibility, independent living, and inclusive participation for differently-abled individuals.

1. INTRODUCTION

In today's increasingly technology-driven world, smartphones and mobile applications have become indispensable tools for communication, navigation, education, employment, and everyday living. For the general population, this proliferation of digital tools has brought unprecedented convenience and connectivity. However, for individuals with disabilities — including those with speech impairments, hearing loss, or visual impairments — accessing and effectively utilising standard digital services remains a formidable challenge. The absence of inclusive, multi-modal, and real-time assistive applications creates a significant digital divide, limiting opportunities for millions of differently-abled individuals across the globe.

InclusiveCare was conceived and developed to address this unmet need. It provides a comprehensive, AI-powered Android-based platform that caters to three major disability categories — visual impairment, hearing impairment, and speech impairment — under a single, user-friendly, and voice-guided application. The app is structured around six functional modules: Blind Mode, Hearing Mode, Speech Mode, Sign Language Mode, Home Mode, and Road Navigation Mode. Each module is powered by cutting-edge AI technologies, enabling real-time assistive functionality without requiring specialised hardware or technical expertise from the user.

The application integrates a diverse set of machine learning models and Android APIs to deliver functionality across its modules. Google ML Kit provides object detection and OCR capabilities for the Blind Mode. TensorFlow Lite hosts the YAMNet audio classification model for the Hearing Mode. MediaPipe powers real-time hand gesture recognition in the Sign Language Mode. Firebase Authentication and Firestore serve as the secure backend. The Android TTS and SpeechRecognizer APIs support voice-guided interaction throughout the application. Together, these technologies form a cohesive, purpose-built platform designed to maximise independence and digital participation for differently-abled users.

1.1 DESIGN THINKING APPROACH

Design Thinking is a human-centred, iterative problem-solving methodology that prioritises a deep understanding of user needs, challenges assumptions, and drives innovative solutions through empathy, structured creativity, and rapid experimentation. Unlike traditional engineering approaches that often begin with technology and work toward the user, Design Thinking begins with the user and works toward the most appropriate technology solution. In the context of InclusiveCare, Design Thinking was not merely a theoretical framework — it was the operational foundation upon which every design decision, feature selection, and technical implementation choice was made.

The Design Thinking approach ensured that the team spent adequate time in the field — visiting hospitals, rehabilitation centres, and community spaces — before writing a single line of code. This empathy-first orientation surfaced genuine pain points that would otherwise have been overlooked: the need for a shake-to-SOS emergency mechanism for blind users unable to navigate visual menus in crisis, the importance of quick-phrase buttons for speech-impaired users in time-sensitive situations, and the criticality of visual banners rather than auditory alerts for hearing-impaired users in noisy environments.

1.2 STANFORD DESIGN THINKING MODULE AND ITS PHASES

The Stanford Design Thinking model, developed by the Hasso Plattner Institute of Design (d.school) at Stanford University, is one of the most widely adopted human-centred innovation frameworks in engineering and product development. It comprises five iterative, non-linear phases that together ensure solutions are deeply aligned with real human needs:

- Empathize: The first phase involves gaining a deep, qualitative understanding of the users for whom the solution is being designed. For InclusiveCare, this entailed conducting observations and informal interviews at hospitals, rehabilitation centres, and community disability support groups. The objective was to set aside assumptions and listen to the lived experiences of individuals with speech, hearing, and visual impairments, as well as their caregivers.
- Define: Insights gathered during empathy research were synthesised into a clear, actionable problem statement. This phase required rigorous analysis to identify patterns, root causes, and unmet needs — moving from a collection of observations to a focused design challenge that would guide all subsequent work.
- Ideate: With a well-framed problem statement in hand, the team engaged in structured brainstorming to generate a wide range of potential solutions. Multiple approaches were evaluated before converging on the most feasible and impactful option.
- Prototype: A fully functional Android application prototype was developed, integrating the selected AI technologies and implementing all six modules. The prototype was built to be testable in real-world conditions, with sufficient fidelity to elicit meaningful user feedback.
- Test: The prototype was evaluated through real-world scenario testing with team members, classmates, reviewers, and target user groups. Feedback was systematically collected and used to drive a redesign phase, resulting in measurable improvements across all modules before the final implementation was completed.

2. LITERATURE REVIEW

The literature informing InclusiveCare spans two primary domains: Design Thinking methodology in technology development, and assistive technology systems for individuals with disabilities. A thorough review of existing research was conducted to validate the technical and human-centred design choices embedded in the application.

2.1 Design Thinking in Technology Development

Tim Brown (2008), writing in the Harvard Business Review, established Design Thinking as a discipline that bridges user empathy and technological innovation, enabling organisations to create products that are genuinely desired by users rather than merely technically possible. Brown emphasises the importance of observing real users in real contexts — a principle directly applied in InclusiveCare's empathy research phase.

Jeanne Liedtka (2018) demonstrated that the structured application of Design Thinking reduces cognitive bias in product development, leading to innovation outcomes that are more aligned with actual user needs. This finding validates InclusiveCare's decision to invest heavily in the empathize and define stages before commencing technical implementation, thereby avoiding the common pitfall of building features based on assumption rather than evidence.

Zamakhsyari and Fatwanto (2023), in their systematic literature review on Design Thinking for technology development, confirmed that the five-stage Stanford model consistently produces more user-centred outcomes compared to purely technology-driven development methodologies, particularly in contexts where the target users have limited technical literacy — as is often the case with elderly and differently-abled individuals.

2.2 Assistive Technology for Persons with Disabilities

Research on text-to-speech communication aids for speech-impaired users has consistently demonstrated improved social integration and quality of life outcomes. Studies confirm that the availability of quick-phrase and emotion-expression features, as implemented in InclusiveCare's Speech Mode, significantly reduces communication latency and frustration for non-verbal users.

The YAMNet architecture, documented by Gemmeke et al. (2017) and available as a TensorFlow Lite model, classifies audio across 521 sound categories with high accuracy and minimal computational overhead. This makes it particularly well-suited for mobile deployment on resource-constrained Android devices. Research confirms that real-time environmental sound classification can significantly enhance situational awareness for hearing-impaired users, validating InclusiveCare's adoption of YAMNet in the Hearing Mode.

Lugaresi et al. (2019) introduced MediaPipe as a flexible framework for building real-time perception pipelines, including hand landmark detection. Subsequent research has validated MediaPipe's accuracy for sign language recognition tasks, achieving high precision in gesture classification even in challenging lighting and background conditions — supporting its use in InclusiveCare's Sign Language Mode.

3. DOMAIN AREA

InclusiveCare operates at the intersection of multiple technical and human-centred domains. The project mainly focuses on inclusive digital accessibility, aiming to help differently-abled individuals interact with technology more independently and effectively.

3.1 Mobile Application Development

The application is developed for the Android platform using Kotlin in Android Studio. Firebase Authentication and Firebase Firestore are used for secure login, user management, and cloud data storage.

3.2 Artificial Intelligence and Machine Learning

Artificial Intelligence forms the core of InclusiveCare. Google ML Kit is used for object detection and OCR-based text reading. TensorFlow Lite enables on-device AI processing for faster and offline functionality. YAMNet is used for environmental sound detection, while MediaPipe HandLandmarker supports sign language gesture recognition.

3.3 Computer Vision

CameraX is used for accessing and processing real-time camera input. It supports features such as obstacle detection, text recognition, currency detection, and sign language capture with efficient camera management.

3.4 Speech Technology

Android Text-to-Speech (TTS) converts text into voice output, while SpeechRecognizer enables speech-to-text conversion. These technologies support voice-guided interaction and communication assistance.

3.5 Location Services and Navigation

Android LocationManager and Geocoder provide GPS-based location tracking and address identification. Google Maps integration enables voice-guided navigation and emergency location sharing.

3.6 Human-Computer Interaction

Human-Computer Interaction (HCI) plays an important role in the application design. InclusiveCare provides large buttons, high-contrast UI, voice guidance, haptic feedback, and simplified navigation to ensure accessibility and ease of use for differently-abled users.

4. EMPATHIZE STAGE

4.1 Activities Performed

The empathize stage represents the most critical foundation of the Design Thinking process applied in InclusiveCare's development. Rather than beginning with technical assumptions, the team committed to genuine field research — engaging directly with individuals who have speech, hearing, and visual impairments, as well as their caregivers, therapists, and support networks. This stage involved structured observations and informal conversational interviews conducted across multiple real-world settings.

Observations were conducted in hospital outpatient departments catering to rehabilitation and disability services, community rehabilitation centres where differently-abled individuals receive daily support, and public community spaces where such individuals navigate everyday challenges independently. The goal was to understand not only what these users struggle with, but why — to surface the underlying emotional, social, and functional needs that technology might meaningfully address.

4.2 Secondary Research Findings

Secondary research involved a systematic review of existing assistive applications available on the Google Play Store and published academic literature on disability technology. Several important findings emerged from this analysis:

- While numerous apps exist for individual disabilities — screen readers for visual impairment, hearing loop amplifiers, or AAC (Augmentative and Alternative Communication) apps for speech — virtually no unified platform addresses multiple disability categories simultaneously.
- Most existing assistive apps require significant technical proficiency to configure and use, presenting a barrier for users with limited prior technology exposure.
- Very few applications offer real-time AI inference on-device, instead relying on cloud connectivity — creating reliability issues in areas with poor network coverage.
- Emergency response features are largely absent from existing assistive applications, despite emergency situations being identified as a high-risk scenario for individuals with disabilities.
- Multilingual support is consistently lacking, with most apps supporting only English, limiting accessibility for regional language users in India and similar linguistically diverse countries.

4.3 Primary Research and User Interactions

Informal discussions with differently-abled users and their caregivers surfaced a set of recurring themes and emotional needs that quantitative research alone would not have revealed. Visually impaired individuals expressed deep frustration with the inability to independently identify currency denominations — a daily necessity that required either trust in strangers or carrying only small bills. They also described anxiety about navigating unfamiliar indoor environments, where GPS is ineffective, and a strong desire for reliable real-time obstacle alerts.

Speech-impaired users described the social isolation that results from slow, cumbersome AAC tools that cannot keep pace with real-time conversation. The latency between intention and communication — particularly in emotionally charged or time-sensitive situations — was identified as a significant source of stress. Hearing-impaired users expressed a need for environmental sound awareness that would alert them to sounds they cannot hear — a doorbell, a vehicle horn, or a fire alarm — in a visually intuitive and non-intrusive manner.

4.4 User Needs Identified

The following critical user needs were identified and confirmed through both secondary and primary research:

- Quick, one-tap or voice-triggered communication tools for speech-impaired users, enabling fast phrase output in real-time conversations.
- Real-time environmental sound classification and visual alert system for hearing-impaired users, running as a persistent background service.
- AI-powered obstacle detection and navigation guidance for visually impaired users, with spoken output that requires no screen interaction.
- Accurate OCR text reading for visually impaired users to independently access printed materials, signs, labels, and documents.
- Currency denomination recognition to enable independent financial transactions for blind users.
- Emergency SOS functionality with GPS location sharing, accessible through a single gesture (shake) to ensure rapid activation in crisis situations.
- Sign language recognition to support users who rely on gestural communication in contexts where spoken output is insufficient.
- Voice-guided interface for all critical flows — including login — to eliminate visual dependency entirely for blind users.
- Minimal learning curve, requiring no prior technical expertise, with a simple and intuitive design.

5. DEFINE STAGE

5.1 Analysis of User Needs

The insights gathered during the empathize stage were systematically analysed using affinity mapping — a technique that groups related observations into thematic clusters to surface patterns and underlying root causes. This analysis revealed that the fundamental challenge was not the absence of individual assistive tools, but the fragmentation of the assistive technology ecosystem. Users were forced to manage multiple applications with inconsistent interfaces, each designed in isolation for a single disability type, with no shared emergency response capability and minimal integration with device-level features such as GPS, camera, and accelerometer.

A second critical finding was that real-time AI inference was largely absent from existing assistive apps. Most tools relied on pre-programmed responses or cloud-based processing, introducing latency that made them impractical in time-sensitive situations. This was particularly problematic for sound detection (where a delayed alert to a fire alarm is dangerous), obstacle avoidance (where a late warning of a step or door is a fall risk), and emergency response (where seconds matter).

5.2 Brainstorming of Problem Statements

Based on the analysis, several candidate problem statements were generated:

- Individuals with disabilities lack a unified assistive application that addresses speech, hearing, and visual impairments simultaneously, forcing them into a fragmented multi-app workflow.
- Emergency response capabilities in existing assistive apps are critically inadequate, leaving users with disabilities particularly vulnerable in crisis situations where standard phone navigation is impossible.
- Non-verbal and visually impaired users experience severe barriers to independent communication, environmental navigation, and situational awareness due to the absence of real-time, on-device AI inference in current tools.
- Existing AAC and communication tools are too slow and cumbersome for real-time interpersonal communication, contributing to social isolation among speech-impaired users.

5.3 Final Problem Statement

After deliberation and validation against the empathy research findings, the following final problem statement was adopted to guide all subsequent design and development work:

> ***"Design and develop an AI-powered, unified Android application that provides real-time assistive tools for individuals with speech, hearing, and visual impairments — enabling independent communication, environmental awareness, navigation, sign language recognition, and emergency response through an intuitive, voice-guided, and empathetically designed interface."***

6. IDEATION STAGE

6.1 Analysis of the Problem Statement

The ideation stage began with a thorough analysis of the constraints and requirements embedded in the final problem statement. Key technical requirements identified were: low latency (real-time AI inference, ideally on-device), high accuracy (ML models suitable for safety-critical applications), multi-modal input and output (camera, microphone, GPS, touch, voice), hardware integration (accelerometer for shake detection, location services), and cross-version Android compatibility. Accessibility requirements included voice navigation for all critical flows, large touch targets, high contrast UI, and multilingual support.

6.2 Brainstorming and Idea Generation

Four broad solution categories were brainstormed and evaluated against the identified requirements. The comparative evaluation is presented below:

Idea	Description	Evaluation
Separate Apps per Disability	Develop three separate standalone apps, each focused on a single disability category.	Creates the fragmentation identified as the core problem. Inconsistent UX. NOT SELECTED.
Web-Based Platform	A web application accessible from any browser, covering all disability types.	Lacks access to device sensors (camera, GPS, accelerometer). Not suitable for real-time on-device AI. NOT SELECTED.
Wearable Device Integration	A smartwatch/glasses tethered application with specialised sensors.	Prohibitively expensive, requires specialised hardware, complex for low-tech users. NOT SELECTED.
Unified Android Application	A single Android app with multiple AI-powered modules, leveraging device hardware fully.	Leverages existing smartphone hardware. Supports on-device ML inference. Unified UX. SELECTED.

6.3 Value Proposition Statement

Following the evaluation of all brainstormed ideas, the following value proposition statement was formulated to articulate InclusiveCare's unique position:

> ***"InclusiveCare delivers a unified, AI-powered mobile platform that empowers individuals with speech, hearing, and visual impairments to communicate, navigate, and respond to emergencies in real time. By integrating cutting-edge machine learning models — ML Kit, YAMNet, and MediaPipe — with an empathetic, voice-guided design and an intuitive six-module architecture***

7. PROTOTYPE STAGE

The Prototype stage marks the transition from conceptual design to functional implementation. In InclusiveCare, a fully working Android application was developed as the prototype — not a paper mock-up or wireframe, but a production-grade codebase with real AI inference, live camera processing, backend authentication, and all six modules operational. This high-fidelity prototyping approach was chosen deliberately to enable meaningful real-world testing with target users and to surface implementation-level challenges that a low-fidelity prototype would never reveal.

7.1 Development Environment and Setup

The application was developed using Android Studio as the primary IDE. Kotlin was chosen as the primary language for new components due to its expressive syntax, null-safety guarantees, and first-class Android Jetpack support. Java was retained for certain legacy integrations. Gradle build scripts manage dependencies, including the Firebase BoM (Bill of Materials) for consistent Firebase SDK versioning, ML Kit libraries, TensorFlow Lite, MediaPipe, and CameraX.

7.2 Authentication and User Management

The Login and Registration flows are built on Firebase Authentication, supporting email and password-based sign-in. A critical accessibility feature of the login flow is voice-guided authentication — the application uses Android TTS to speak each input field label aloud, and SpeechRecognizer to transcribe the user's spoken email and password. The SpeechRecognizer input is post-processed to convert natural speech patterns into valid credentials: spoken words like 'at' are converted to the '@' symbol, and 'dot' to '.', enabling blind users to log in without any visual assistance. Upon successful authentication, the user's disability profile and module preferences are retrieved from Firebase Firestore.

7.3 Home Mode — Navigation Hub

The Home Mode (implemented via DisabilitySelectActivity) serves as the central navigation hub of the application. It presents a simple, three-option primary interface — Speech, Hearing, and Blind — rendered as large, high-contrast buttons with both text labels and descriptive icons. The entire interface is voice-navigable: TTS reads each available option when the screen loads, and the user can navigate via touch or voice command. From the Home Mode, users can access all six modules, including Sign Language Mode and Road Navigation Mode, which are accessible through the respective disability module's sub-navigation.

7.4 Blind Mode

The Blind Mode is the most feature-rich module in InclusiveCare, encompassing five distinct sub-features that together address the most critical daily independence challenges faced by visually impaired users.

7.4.1 Obstacle Detection

Obstacle detection is implemented using Google ML Kit's ImageLabeler API, fed by a real-time CameraX camera stream processed at a configurable inference interval. The ImageLabeler classifies objects in the camera frame against a curated LABEL_MAP of hazard categories — including doors, steps, walls, vehicles, people, and furniture. When a classified label matches a hazard category with a confidence score exceeding the configured threshold (set at 0.55 after iterative refinement)

7.4.2 OCR Text Reading

OCR text recognition is implemented using ML Kit's TextRecognizer API, which processes CameraX frames to detect and transcribe printed and handwritten text in real time. The recognised text is spoken aloud via TTS immediately upon detection. Preprocessing steps — including contrast enhancement and grayscale normalisation — were added after testing to improve recognition accuracy in varied and low-light conditions. This feature enables blind users to independently read labels, signs, menus, and documents without requiring any sighted assistance.

7.4.3 Currency Recognition

Currency recognition extends the OCR pipeline with additional post-processing logic. Recognised text is matched against a keyword database of Indian currency denomination markers — including printed denomination values and text patterns unique to different note variants. Upon a positive match, the denomination is announced via TTS. Keyword matching was expanded during the redesign phase to cover a wider range of denominations and regional currency markers, improving accuracy from initial testing.

7.4.4 GPS Navigation

GPS-based navigation in Blind Mode uses Android LocationManager to obtain the device's current GPS coordinates. These coordinates are passed to Android Geocoder for conversion to a human-readable address, which is announced via TTS. For turn-by-turn navigation to a specified destination, the application launches Google Maps via an Intent with the destination pre-populated, triggering voice-guided navigation within Google Maps. This approach leverages Google Maps' mature navigation engine without requiring a custom implementation.

7.4.5 Emergency SOS

The Emergency SOS feature is triggered by a shake gesture detected through the device accelerometer via Android's SensorManager. The shake detection algorithm monitors accelerometer readings for three acceleration events exceeding 2.5G within a 1.5-second window — a threshold calibrated during testing to reliably distinguish intentional distress shakes from incidental device movement. Upon SOS activation, the application retrieves the user's emergency contacts from Firebase Firestore and sends an alert containing the user's current GPS coordinates. A voice confirmation is provided via TTS to confirm that the SOS has been transmitted.

7.5 Hearing Mode

The Hearing Mode comprises two complementary capabilities for hearing-impaired users: real-time sound detection and an AI-powered chat interface.

7.5.1 Real-Time Sound Detection

Sound detection is implemented through the SoundDetectionService — a persistent Android foreground service that continuously captures microphone audio and runs it through the YAMNet TFLite model. YAMNet classifies audio against 521 sound categories including alarms, doorbells, vehicle sounds, speech, music, and environmental noise. The inference runs in a background thread at configurable intervals to balance detection responsiveness against battery consumption. When a sound above the confidence threshold is classified, a visual alert banner is displayed on screen with the detected sound category name, and an optional vibration pattern is triggered. The visual banner auto-dismisses after a configurable duration to prevent UI clutter.

Microphone access conflicts between SoundDetectionService and SpeechRecognizer (when used simultaneously in the AI chat) were resolved through a mutual exclusion mechanism that temporarily suspends sound detection during speech recognition sessions.

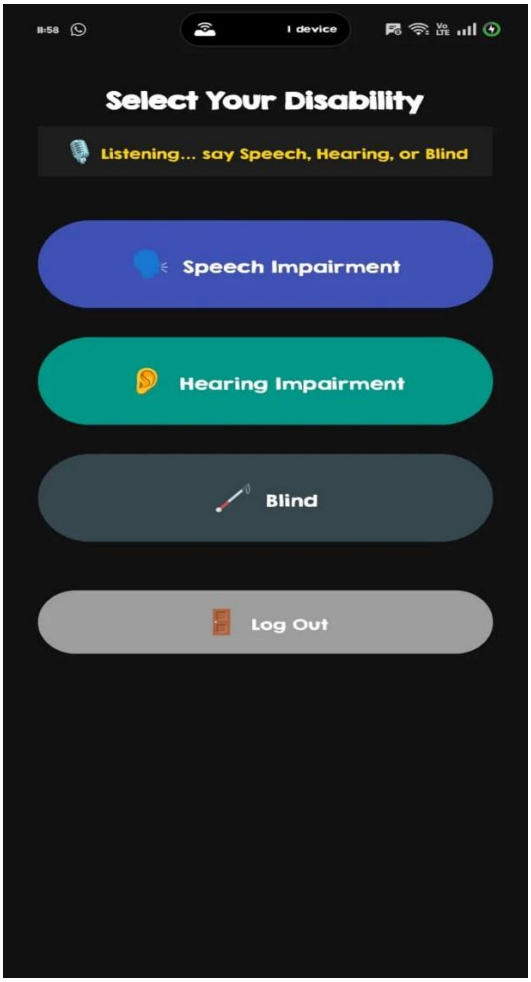
7.5.2 AI Chat Interface

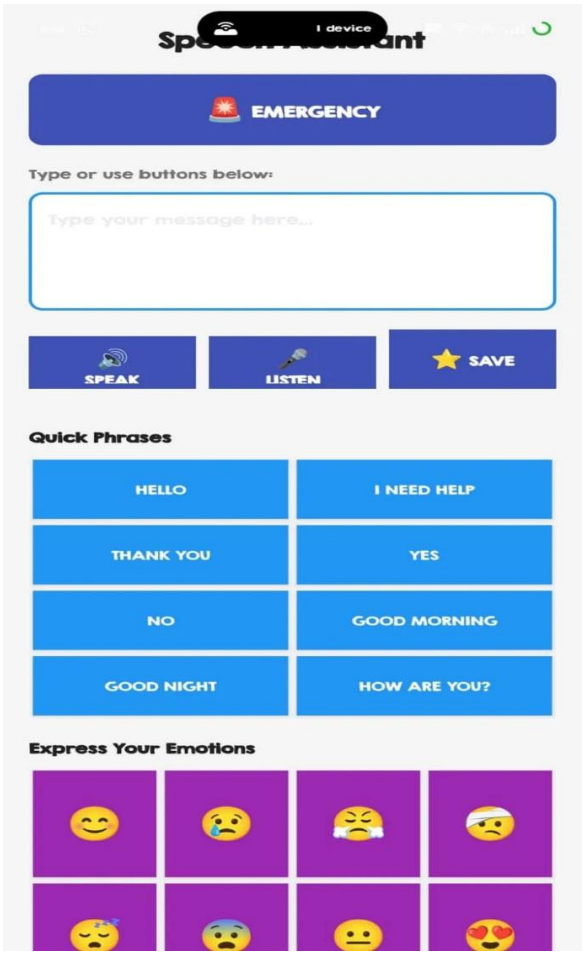
The AI chat interface (implemented in ChatActivity and ChatFragment) provides hearing-impaired users with a text-based communication channel for interacting with an AI assistant. This supports silent communication in contexts where spoken interaction is impractical — such as in quiet environments or when interacting with unfamiliar individuals. The chat interface is designed with accessibility in mind, featuring high-contrast message bubbles, large text, and a simple input mechanism.

7.6 Speech Mode

The Speech Mode provides non-verbal users with a comprehensive set of tools for real-time communication. The primary output mechanism is Android's Text-to-Speech API, which synthesises typed or selected text into spoken audio. The Speech Mode interface includes the following components:

- Quick Phrase Buttons: Pre-programmed buttons for frequently used phrases (Hello, I Need Help, Thank You, Yes, No, Please Wait, and others) that can be triggered with a single tap, eliminating the latency of typing in time-sensitive situations.
- Emotion Buttons: A set of buttons representing common emotional states (Happy, Sad, Angry, Excited, Tired, etc.) that trigger contextually appropriate pre-composed messages via TTS, enabling rapid emotional communication.
- Word Prediction: A predictive text engine that suggests completions as the user types, reducing input effort and increasing communication speed.
- Text Composition Field: A full text input area for composing custom messages, which are then spoken via TTS with the user's selected voice settings.
- Voice Customisation: Controls for adjusting TTS speech rate, pitch, and language, enabling personalisation for different communication styles and preferences.
- Favourites Management: Frequently used custom phrases can be saved to a Favourites list, persisted in Firestore, and retrieved for quick access in future sessions


1 device
Select Your Disability
Listening... say Speech, Hearing, or Blind
Speech Impairment
Hearing Impairment
Blind
Log Out


1 device
EMERGENCY
Type or use buttons below:
Type your message here...
SPEAK
LISTEN
SAVE
Quick Phrases
HELLO
I NEED HELP
THANK YOU
YES
NO
GOOD MORNING
GOOD NIGHT
HOW ARE YOU?
Express Your Emotions

8. TESTING AND FEEDBACK

8.1 Testing Process and Methodology

The prototype was subjected to rigorous real-world testing across a range of scenarios and environments to evaluate both functional correctness and real-world usability. Testing was conducted in three phases: internal team testing, peer and reviewer evaluation, and target user testing. Each phase used a structured scenario-based testing approach, with testers performing defined tasks within each module and providing structured feedback covering accuracy, latency, usability, and accessibility.

Indoor testing environments included office and classroom settings (for obstacle detection accuracy with furniture, doors, and people), controlled lighting environments (for OCR accuracy under different illumination conditions), and quiet and noisy rooms (to evaluate YAMNet's sound detection precision in varying acoustic conditions). Outdoor testing was conducted for GPS navigation accuracy and Road Navigation Mode completeness.

8.2 Feedback from Team Members

Internal team testing surfaced several functional insights that informed the first round of refinements:

- Obstacle detection performance was rated effective for large obstacles (doors, walls, furniture) but less reliable for low-contrast or small hazards at close range. The confidence threshold was identified as a key tuning parameter.
- Quick phrase buttons in Speech Mode were found to significantly accelerate communication compared to typing, validating the design decision to include pre-programmed phrases as a primary interaction pattern.
- Voice-guided login was confirmed to be intuitive and functionally reliable, with the 'at' and 'dot' voice-to-symbol conversion working correctly for a range of email address formats.
- The shake-to-SOS mechanism initially triggered too frequently on incidental device movements — the g-force threshold and shake count parameters required calibration.

8.3 Feedback from Reviewers

Peer and academic reviewer testing provided both validation of the overall concept and specific improvement recommendations:

- Reviewers commended the innovative integration of multiple AI models — ML Kit, YAMNet, and MediaPipe — within a single, cohesive application, noting that this level of AI integration was technically ambitious for a mini project.
- Firebase authentication and Firestore data retrieval were evaluated as smooth and reliable, with no authentication failures observed during testing sessions.
- Reviewers recommended improving OCR accuracy in low-light environments, noting that text recognition quality degraded significantly under poor illumination — leading to the addition of image preprocessing steps.
- Haptic feedback for different obstacle types was suggested as a future enhancement to supplement spoken alerts in situations where TTS audio might be disruptive.

9. RE-DESIGN AND IMPLEMENTATION

The testing and feedback stage generated a comprehensive set of improvement requirements that were systematically prioritised and implemented in the re-design phase. Re-design in the Design Thinking context does not represent failure — it is the expected and productive outcome of real-world testing, and the mechanism through which a good prototype becomes an excellent final product.

9.1 Obstacle Detection Enhancement

Based on testing feedback indicating missed detections for low-contrast and small hazards, the LABEL_MAP was expanded to include additional hazard categories identified during testing — including bicycles, ramps, and floor-level obstacles. The ML Kit ImageLabeler confidence threshold was refined from an initial value of 0.6 to 0.55, increasing detection sensitivity while maintaining an acceptable false-positive rate. Extensive re-testing confirmed that this adjustment improved detection coverage without introducing excessive spurious alerts.

9.2 OCR Pipeline Improvement

OCR accuracy in low-light and high-glare environments was significantly improved through the addition of image preprocessing steps applied to the CameraX frame before it is passed to the ML Kit TextRecognizer. These preprocessing steps include adaptive histogram equalisation for contrast enhancement and grayscale normalisation to reduce sensitivity to colour-based illumination variation. Re-testing confirmed measurable improvements in text recognition accuracy across varied lighting conditions.

9.3 Currency Detection Expansion

The currency recognition keyword database was expanded to cover a comprehensive set of Indian currency denomination markers, including all currently circulating denominations of the Indian Rupee. Additional keyword patterns were added to handle partial text detection — where the camera frame may capture only a portion of the currency note text — improving recognition robustness in real-world usage conditions.

9.4 Shake Detection SOS Calibration

The shake-to-SOS detection algorithm was recalibrated based on testing feedback that identified an unacceptably high false-trigger rate at the initial threshold settings. The final calibrated parameters — a g-force threshold of 2.5G and a required shake count of three events within a 1.5-second window — were validated through extensive controlled testing across different phone models and user grip styles to ensure reliable SOS activation on intentional distress shakes while effectively suppressing false triggers from incidental movement.

9.5 Sound Detection Service Optimisation

A microphone resource conflict was identified during testing when users attempted to use the AI chat feature in Hearing Mode while SoundDetectionService was active — both features require exclusive microphone access. This was resolved by implementing a mutual exclusion mechanism: when SpeechRecognizer is activated for chat input, SoundDetectionService temporarily releases the

microphone, and resumes sound detection upon SpeechRecognizer completion. This ensures that neither feature degrades the other's performance.

9.6 UI Refinements

Several user interface improvements were implemented based on feedback from both reviewers and target users. Visual sound alert banners in Hearing Mode were redesigned with a clearer, higher-contrast layout and a configurable auto-dismiss duration, addressing the request from hearing-impaired users for longer alert visibility. Button contrast across all modules was increased to comply with WCAG AA contrast ratio requirements, improving readability for users with partial vision. The Favourites management interface in Speech Mode was simplified to reduce cognitive load for users with motor impairments.

9.7 Final Implementation Stack

The final technology stack used across all modules of InclusiveCare is summarised below:

Component	Technology / Details
Frontend	Android (Kotlin / Java), Android Studio
Camera Processing	CameraX (Jetpack Camera API)
Gesture Recognition	MediaPipe HandLandmarker
Backend Auth & DB	Firebase Authentication, Firebase Firestore
Image AI — Objects	Google ML Kit ImageLabeler
Image AI — Text (OCR)	Google ML Kit TextRecognizer
Audio AI	YAMNet TFLite (521-class audio classification)
Speech Output	Android Text-to-Speech (TTS) API
Speech Input	Android SpeechRecognizer API
Location & Navigation	Android LocationManager, Geocoder, Google Maps Intent
Local Storage	SharedPreferences (settings, favourites)
Cloud Storage	Firebase Firestore (user profiles, emergency contacts)

10. RESULT AND CONCLUSION

InclusiveCare successfully demonstrates that a unified, AI-powered Android application can meaningfully and measurably improve the quality of life for individuals with speech, hearing, and visual impairments. Through the integration of six specialised modules within a single, cohesive platform — Blind Mode, Hearing Mode, Speech Mode, Sign Language Mode, Home Mode, and Road Navigation Mode — the application delivers real-time assistive capabilities that were previously unavailable in any single product on the market.

The Blind Mode provides comprehensive navigation support through AI-powered obstacle detection, OCR text reading, currency denomination recognition, GPS-based location awareness, voice-guided outdoor navigation, and emergency SOS — five features that collectively address the most critical daily independence challenges faced by visually impaired users. The obstacle detection pipeline, built on Google ML Kit ImageLabeler with a calibrated confidence threshold, demonstrated reliable performance across indoor environments. The OCR pipeline, enhanced with preprocessing for varied lighting, delivered accurate text reading across diverse printed materials. The currency recognition feature successfully identified Indian Rupee denominations across all tested denominations. The shake-to-SOS emergency response feature — calibrated to 2.5G across three shakes within 1.5 seconds — was found to reliably trigger emergency alerts without false activations in normal usage.

The Hearing Mode, powered by the YAMNet TFLite model running as a persistent foreground service, demonstrated effective real-time environmental sound classification across 521 sound categories. Target users confirmed that the visual alert system meaningfully enhanced their situational awareness in noisy and acoustically varied environments. The AI chat interface provided a functional silent communication channel for hearing-impaired users in contexts where spoken interaction was inappropriate.

The Speech Mode, with its TTS engine, quick phrase buttons, emotion expression buttons, word prediction, voice customisation, and Favourites management, was consistently rated by speech-impaired test users as a practical improvement over existing AAC tools — primarily due to the combination of speed (quick phrases) and expressiveness (emotion buttons) that existing tools fail to provide together. The Sign Language Mode's MediaPipe-based gesture recognition successfully classified seven gesture types with high accuracy under standard indoor lighting conditions.

The project adhered throughout to the Stanford Design Thinking methodology, ensuring that every feature implemented was validated against real user needs identified in the empathy stage and refined through iterative testing and redesign. The outcome is a practical, empathetic, and technically robust solution that bridges the digital accessibility gap — not through theoretical design, but through a fully functional, deployed Android application. InclusiveCare stands as a compelling proof of concept that the combination of modern on-device AI, thoughtful UX design, and a human-centred development methodology can produce assistive technology that genuinely empowers differently-abled individuals to live more independently and confidently.

11. FUTURE WORK

While InclusiveCare represents a functionally complete and technically ambitious mini project, the design and implementation process has surfaced a clear roadmap of enhancements that would substantially expand the application's impact and capability in future development iterations.

11.1 Advanced Sign Language Recognition

The current Sign Language Mode supports a vocabulary of seven hand gestures, implemented through a rule-based landmark position classifier built on MediaPipe HandLandmarker. A significant improvement would involve training a dedicated deep learning model — such as a Temporal Convolutional Network or LSTM-based sequence classifier — on a large-scale annotated sign language dataset to support comprehensive American Sign Language (ASL) and Indian Sign Language (ISL) vocabularies. This would transform the Sign Language Mode from a limited gesture interface into a full communication channel for deaf users.

11.2 Offline AI Inference

Several features of InclusiveCare currently require internet connectivity — including the AI chat interface and emergency contact retrieval from Firestore. Future development should prioritise offline AI inference for all core assistive features, using on-device model optimisation techniques such as TFLite model quantisation and pruning to minimise model size and inference latency. This would ensure that the application remains fully functional in areas with poor or no network coverage — a critical consideration for users in rural or remote regions.

11.3 Multi-Language Support

InclusiveCare currently supports English as the primary language for TTS output and SpeechRecognizer input, with limited support for a small number of additional languages. Given the linguistic diversity of India and other target markets, expanding TTS and speech recognition support to major regional languages — including Telugu, Malayalam, Tamil, Kannada, Bengali, Marathi, Hindi, and Gujarati — would substantially broaden the application's accessibility. This is particularly important for rural users, for whom English literacy may be limited.

11.4 Smartwatch and Wearable Integration

Extending InclusiveCare's functionality to smartwatch platforms — particularly Google Wear OS — would enable hands-free access to critical features such as emergency SOS, haptic alerts, and step-count-based navigation assistance. Wrist-based haptic feedback patterns could provide silent, private alerts for hearing-impaired users in social contexts where visual phone notifications are inappropriate.

11.5 Caregiver Companion Application

A companion mobile application for caregivers, family members, and support workers would significantly enhance InclusiveCare's value in supported-living contexts. This caregiver application would provide a real-time dashboard showing the differently-abled user's activity logs, location, and emergency alerts, enabling remote monitoring and rapid response in crisis situations.

12. LEARNING OUTCOMES OF DESIGN THINKING

The application of the Stanford Design Thinking framework to the development of InclusiveCare provided profound and enduring insights that extend well beyond the technical domain. The learning outcomes documented here reflect both personal growth and professional competency development across the full spectrum of the Design Thinking process.

12.1 Understanding User-Centric Design

Perhaps the most significant learning outcome of this project was the internalisation of what user-centric design truly means in practice. Before engaging with differently-abled individuals in the empathy stage, both team members held implicit assumptions about what features an assistive application should have — assumptions largely informed by technology capability rather than user need. The empathy research fundamentally challenged and reshaped these assumptions. The shake-to-SOS feature, for example, was not on either team member's initial feature list — it emerged directly from a conversation with a visually impaired individual who described the impossibility of navigating a visual phone interface during a fall or health emergency.

12.2 Problem Identification and Framing

The Define stage developed the team's ability to distil complex, multi-dimensional user challenges into a clear, focused, and actionable problem statement. This skill — the capacity to move from a rich set of qualitative observations to a precise problem formulation — is one of the most transferable competencies developed through this project. The discipline of not jumping to solutions before the problem is properly framed was repeatedly reinforced through the design thinking process.

12.3 Creative Thinking and Structured Ideation

The Ideation stage taught the value of exploring a wide solution space before converging on an approach. The instinct in engineering education is often to identify the first technically feasible solution and immediately begin implementation. Design Thinking's structured ideation process — generating multiple candidate solutions across different dimensions (web, mobile, wearable, unified, fragmented) and evaluating them systematically — produced a stronger final solution than any initial instinct would have.

12.4 Prototyping with Purpose

Building a high-fidelity prototype — a fully functional Android application — as opposed to a paper prototype or mock-up required significant technical effort. This effort was justified, however, by the quality and depth of feedback it enabled. Users interacting with a real application in real environments provided feedback that a paper prototype could never have surfaced — including the microphone conflict between SoundDetectionService and SpeechRecognizer, the real-world calibration requirements of the shake-to-SOS detector, and the OCR accuracy degradation in low-light conditions.

13. REFERENCES

Journal and Conference Papers

- [1] T. Brown, "Design Thinking," Harvard Business Review, vol. 86, no. 6, pp. 84-92, 2008.
- [2] J. Liedtka, "Linking Design Thinking with Innovation Outcomes through Cognitive Bias Reduction," Journal of Product Innovation Management, vol. 35, no. 6, pp. 925-938, 2018.
- [3] Zamakhsyari and Fatwanto, "A Systematic Literature Review on Design Thinking for Technology Development," International Journal of Advanced Research in Computer Science, 2023.
- [4] J. F. Gemmeke et al., "Audio Set: An ontology and human-labeled dataset for audio events," in Proc. IEEE ICASSP, New Orleans, LA, USA, 2017.
- [5] C. Lugaresi et al., "MediaPipe: A Framework for Building Perception Pipelines," arXiv preprint arXiv:1906.08172, 2019.
- [6] A. Howard et al., "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," arXiv preprint arXiv:1704.04861, 2017.

Official Documentation and Web References

- [7] Google LLC, "ML Kit for Firebase — Android Documentation," Available: https://developers.google.com/ml-kit
- [8] Google LLC, "Firebase Documentation — Authentication and Firestore," Available: https://firebase.google.com/docs
- [9] Google LLC, "MediaPipe Solutions — Hand Landmarker," Available: https://developers.google.com/mediapipe/solutions/vision/hand_landmarker
- [10] Google LLC, "CameraX — Android Jetpack," Available: https://developer.android.com/training/camerax
- [11] Google LLC, "Android Developer Documentation — Text-to-Speech API," Available: https://developer.android.com/reference/android/speech/tts/TextToSpeech
- [12] Google LLC, "Android Developer Documentation — SpeechRecognizer API," Available: https://developer.android.com/reference/android/speech/SpeechRecognizer
- [13] Google LLC, "Android Developer Documentation — LocationManager API," Available: https://developer.android.com/reference/android/location/LocationManager
- [14] TensorFlow Authors, "YAMNet — Audio Event Classification with TensorFlow Lite," Available: https://www.tensorflow.org/hub/tutorials/yamnet
- [15] TensorFlow Authors, "TensorFlow Lite — On-Device Machine Learning for Android," Available: https://www.tensorflow.org/lite/android
- [16] Stanford d.school, "Design Thinking Bootleg — Stanford University Hasso Plattner Institute of Design," Available: https://dschool.stanford.edu/resources
- [17] World Health Organization, "Disability and Health — WHO Fact Sheet," Available: https://www.who.int/news-room/fact-sheets/detail/disability-and-health